

Real time factor 1 ise gercek dünya esit

0.6 ise gercek dünyada 10 sn ise simülasyonda 6 sn dır

Odometri: Robotun bir t andan bir $t+1$ ana gecerken kullanılan hızlar, linear x - linear y - angular z. $\underbrace{x-y-\theta}_{\text{konum}} \quad \underbrace{x-y-\theta}_{\text{hız}}$
odometrinin dizisiyle ^{robotun} izine (t-trajectory) Pose + hız

ulaşabiliriz. ^{gidecek yol path} ^{gidilen yol trajectory} ~~odometri~~

lokalisasyon: Nerede ^{harita ve yerimizi veriyor} olduğunu takip edebilmek

lokale olmak: Harita verilir ve haritada nerede olduğunu bulmak. Dönerken konum ⁰ kaybedilebilir, ^{simülasyon}

Kaçırılmış robot problemi

Haritalama: SLEM $\rightarrow$ Haritalama + lokalisasyon

Navigasyon: Şu anki konumdan X konumuna robotun ^{otonom} gitmesi işlemi

Exploration (keşif): Navigasyonla farkı robotun gideceği yere kendi karar vermesi

termal base exploration: Termal bir görüntü görene kadar keşif yapmak.

3D Haritalama: Drenel haritalama

07.03.22
2.

raw data $\rightarrow$ yorumlanmamış bilgi $\rightarrow$ laser mesajı önce 10 sonra 9 m oluyor. Bu raw data bize yaklaşan bir şey var demek information.

Path execution $\rightarrow$ Sınıtlama ve dinamik engeller vs. zer

$\rightarrow$ eğerleli

Wheeled mobile robots

• Locomotion: Robot nasıl hareket ediyor

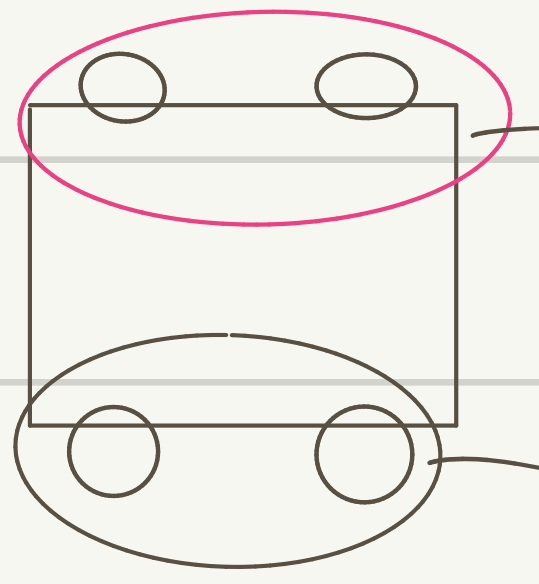
• Sensing:

• Control: Robot nasıl tasarlanmış?

• Reasoning: Robottan aldığın bilgilerle haritaya doğru mu? vs.

• Communication: Monitörle robot - robotla robot vs.

Diferansiyel sürüşlü robotlar her teker ayrı yöne dönebilir, özel tekerleği vardır, yan yan gidebilir

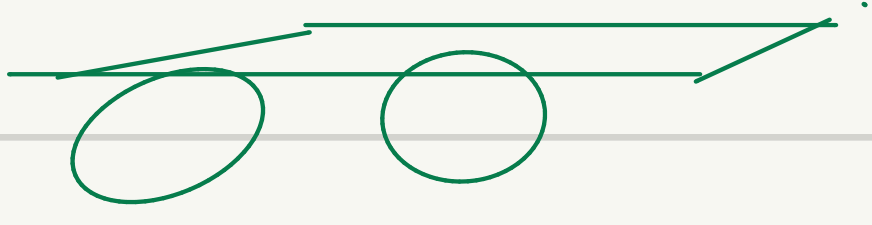


Böyle hızlanırlar, kontrolü zor.

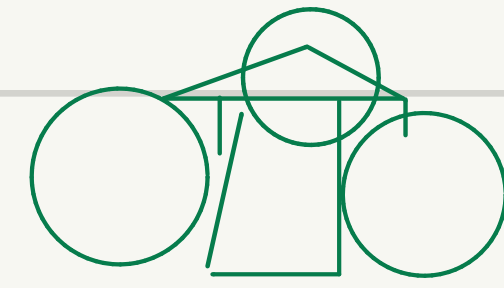
16.03.22
3.

WMR

bi-wheel type robot

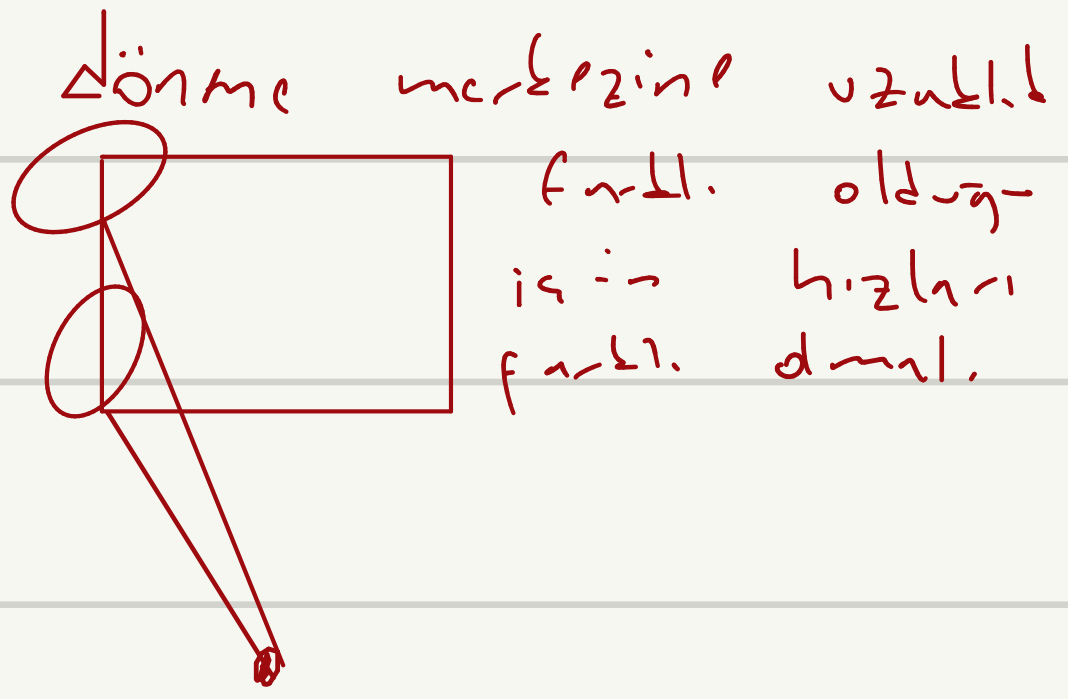


caterpillar type robot



omnidirectional robot

çünkü az sayıda tekerlek



2D translation



2D rotation →

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

harita bilisi-
 lokalizasyon $\rightarrow$ B haritada robotun nerede olduğunu anlama çabası.

Öğütme $\rightarrow$ Bluetooth sensörü ile gerçeğe yakın konum verir.

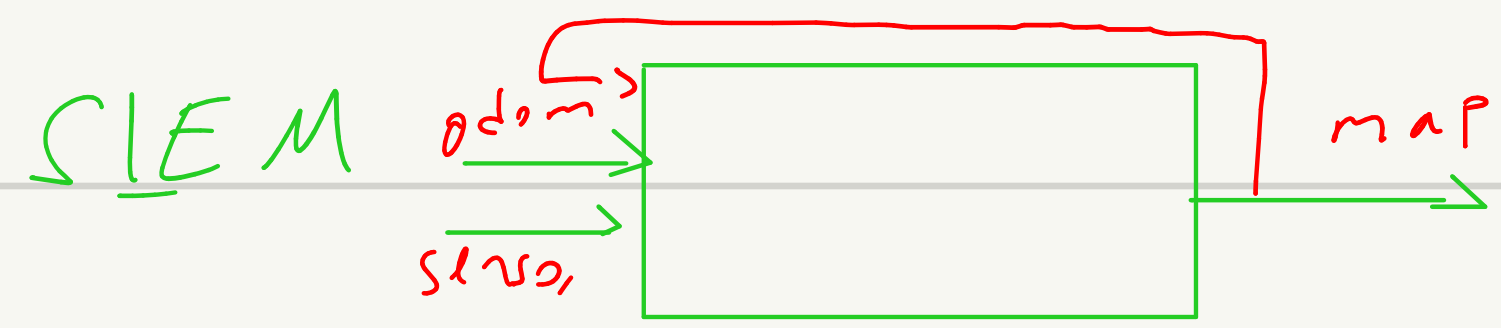
Trajectory $\rightarrow$ O'dan E'ye konum dizisi

Motion Models $\rightarrow$ Odometry $\rightarrow$ Tekleğin açısını falan bilir
 $\hookrightarrow$ Velocity based

Visual odometry $\rightarrow$ Kamera aracılığıyla.

Kayıp robot problem

Landmark $\rightarrow$ Temel aldığı yön nesnesi.



06.04.22
5.

Pointcloud $\rightarrow$ 3 boyutlu lazer verisi

gideceğiniz yol path
giçtiğiniz yol trajectory

Navigation

Static map $\rightarrow$ Önceden harita çıkarmışsak konumunuz + harita geliyor.

Holonomic $\rightarrow$ Omni directional $\rightarrow$ her yere gidebilir

Navigation planners $\rightarrow$ Bulduğunuz konumdan hedefe en kısa yolu çıkarır.

local planners - global planners

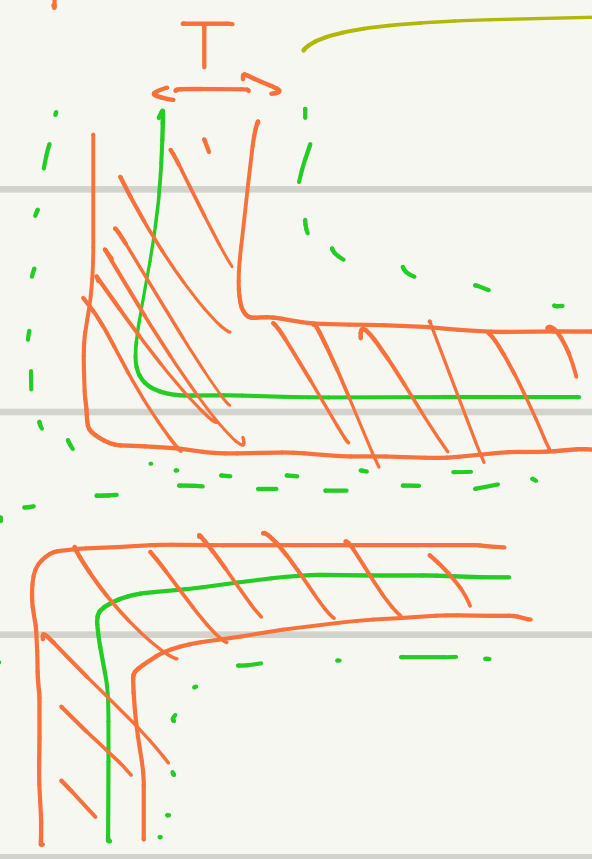
↓
ara hedefler

goal tolerance $\rightarrow$ hata toleransı $\approx 2m$ hata ile git

trajectory scoring $\rightarrow$ global pathle örtüşme oranı ne kadar olsun

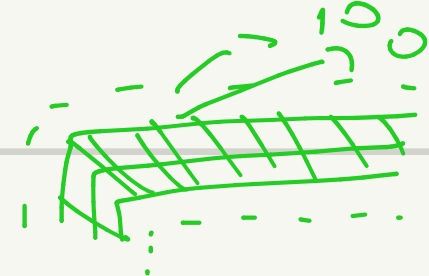
cost map $\rightarrow$ Verilen bir parametre ölçüsünde engeller kalınlaştırılıyor
 $\rightarrow$ Sorunun dedi
cost mapın genişliği nasıl belirlenir?

Eğer mapten çıkıyorsa
Global costmap
lazer verilerinden vs.
çıkıyorsa "local costmap"
Mesela hareketli insan



recovery behavior
 $\rightarrow$ robot costmapta
engelle değışken çakışma
çakışma çakışır.

inflation radius $\rightarrow$ robot burada olabilir
ama buradan uzağa olmaya çalışır
çünkü duvarın sürebilir skorlaması
vardır.



11.05.22
7.

Görüntü işleme

3 boyutlu görüntü için 2 kamera ve kameranın arasındaki uzaklık bilgisi gerektirir, derinlik sadece kameraların kesiştiği noktalarda kestirilebilir.

Kameralar balık gözü gibi görüyor $\rightarrow$ rectifier ile lens distortion düzelebilir

RGBD $\rightarrow$ derinlik kamerası

tek kameralarla t anında ve $t+1$ anında görüntü alarak derinlik algılanabilir. $\hookrightarrow$ popüler ve yaygın çünkü hafif

OPTIC FLOW $\rightarrow$ Yansımanın ve ışık akışının nasıl değiştiğini algılayan sensör, insan gözü için anlam ifade etmez, donelerin altına 90° yerleşir ve yerden yükseklik - stabilizasyonu gibi işleri yapar. Motion field slaytı var.

Objeye yakınsa motion hızlı akar.

Color tracking $\rightarrow$ Sarı rengi 1 diğerlerini 0 yapar nasıl

Edge extraction $\rightarrow$ kenar bulmaya yarar

Corner detection $\rightarrow$ Radikal değişiklik işaretler

Not: işleme girmedi sadece $\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$ 'e girer

Blob detection $\rightarrow$ Yakın renkleri kümelendir, blok blok küller

ve büyüklüğe - geometriye göre filtreler. Ya da alan filtreler. Örneğin plaka bulacak, ormanı filtreler.

18.07.22
8.

Proje → Exploration 4

Navigation ve kesif için hazır kod kullan

3D haritalama → Engeli arazi

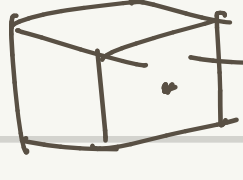
↳ Problemler → kesinlik - hafıza - zaman

↳ 2D'ye göre

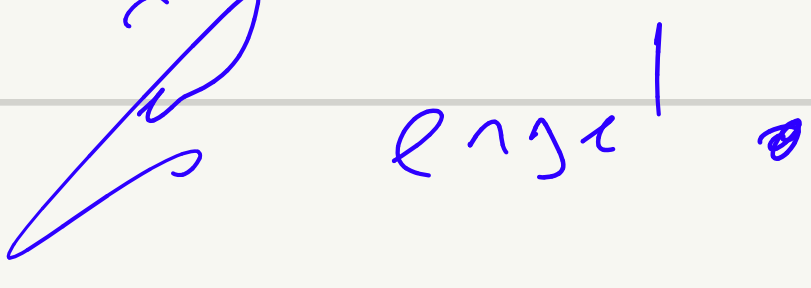
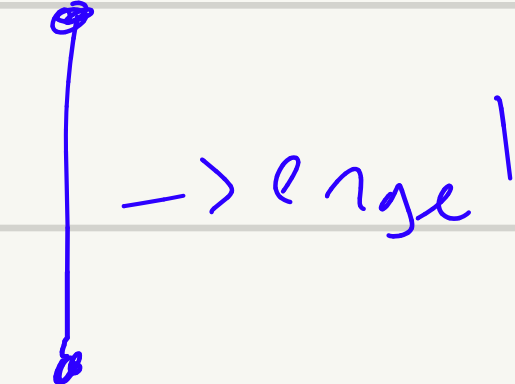
↳ Holux 1080 tane lazer atıyor
44 byte 160 kByte olur 1s'de

↳ intel'de 160 MByte → işleme - depolama zor
↳ sıkıştırarak ya da örneklemi azaltarak çözülür

Voxel Grid → Aynı değere sahip ardışık değerler 1 değer

gibi tutulur. Gridlerde Gözetimlik var.  → hepsi engel sayılır.

Elevation map → 2.5D → kesik kesik 2 boyutlu haritaları birleştirip oluşturulur. Geçişler 3D harita kadar yumuşak değil

 engel → 

Point Cloud → Grid, göre daha yoğun ve detaylı. Tarsan,

Oct-ee → Voxel